

# Rapport de déploiement

Application TaskFlow sur serveur Hetzner Cloud

|              |                                                                                                     |
|--------------|-----------------------------------------------------------------------------------------------------|
| Projet       | TaskFlow — gestion de tâches full-stack                                                             |
| Auteur       | Hamza Hamarrass (avec l'assistance de Claude Code)                                                  |
| Date         | 4 octobre 2026                                                                                      |
| URL publique | <a href="https://37-27-147-21.sslip.io">https://37-27-147-21.sslip.io</a>                           |
| Serveur      | Hetzner Cloud CX23 — ubuntu-4gb-hel1-2 — Helsinki — 37.27.147.21                                    |
| Dépôt GitHub | <a href="https://github.com/HAMARRASS-lab/TaskFlow-">https://github.com/HAMARRASS-lab/TaskFlow-</a> |
| Statut       | En ligne, HTTPS actif, tests réussis                                                                |

## 1. Résumé

L'application **TaskFlow** (API Spring Boot + interface Angular + base PostgreSQL) a été déployée sur un serveur virtuel Hetzner Cloud à l'aide de Docker Compose. Le site est servi en **HTTPS** avec un certificat Let's Encrypt renouvelé automatiquement par Caddy. Le déploiement est reproductible en une seule commande grâce au script `deploy.sh`, versionné sur GitHub. Des tests fonctionnels de bout en bout (inscription, connexion, création, modification, changement de statut et suppression de tâches) ont été réalisés en direct sur la version HTTPS et sont tous passés.

| Élément                                                                | Résultat |
|------------------------------------------------------------------------|----------|
| Serveur provisionné et accessible en SSH (clé <code>stackflow</code> ) | OK       |
| Docker installé et stack de production démarrée (4 conteneurs)         | OK       |
| Secrets générés sur le serveur, hors du dépôt Git                      | OK       |
| HTTPS Let's Encrypt + redirection HTTP → HTTPS                         | OK       |
| Tests de bout en bout dans le navigateur (HTTP puis HTTPS)             | OK       |
| Code de déploiement commité et poussé sur GitHub                       | OK       |
| Nom de domaine personnalisé (DuckDNS)                                  | En cours |

## 2. Architecture

| Couche          | Technologies                                                                      |
|-----------------|-----------------------------------------------------------------------------------|
| Backend         | Java 21, Spring Boot 3.3, Spring Security + JWT, Spring Data JPA, Flyway, OpenAPI |
| Frontend        | Angular 18 (standalone), NgRx, Angular Material, servi par nginx                  |
| Base de données | PostgreSQL 16 (volume Docker persistant <code>pgdata</code> )                     |
| Proxy HTTPS     | Caddy 2 — certificat Let's Encrypt automatique, HTTP/2 et HTTP/3                  |
| Infrastructure  | Hetzner Cloud CX23 (2 vCPU x86, 4 Go RAM, 40 Go disque), Ubuntu 26.04 LTS         |
| Outils          | Docker Compose, rsync/SSH, GitHub                                                 |

### Flux d'une requête

```
Navigateur —HTTPS:443—> Caddy —HTTP—> frontend (nginx :80)
                                |— fichiers Angular (/ , /tasks, ...)
                                |— /api/* —> backend (Spring Boot :8080)
                                    |—> db (PostgreSQL :5432)
```

Seuls les ports 80 et 443 (Caddy) sont exposés sur internet.  
Le backend et la base de données restent sur le réseau Docker interne.

### 3. Étapes réalisées

#### Étape 1 — Préparation de l'environnement

- Démarrage du navigateur Chromium conteneurisé (`docker-browser`, accessible sur `http://127.0.0.1:3000`) utilisé pour toutes les actions web.
- Localisation du projet dans `~/IdeaProjects/taskflow` et analyse : backend Spring Boot, frontend Angular, fichiers Docker, README.
- Constat : l'application tournait en local en mode développement (Angular sur le port 4200, API sur 8080, PostgreSQL dans Docker).

#### Étape 2 — Identification du serveur Hetzner

- Ouverture de la console Hetzner (projet *task-flow*) dans le navigateur Docker ; connexion effectuée par l'utilisateur.
- Serveur identifié : **ubuntu-4gb-hel1-2**, type CX23, datacenter Helsinki, IP publique **37.27.147.21**.
- Clé SSH `stackflow` présente dans le projet Hetzner (empreinte vérifiée : `48:1b:42:....c5:e3`, identique à `~/ssh/stackflow.pub`).

#### Étape 3 — Résolution du problème d'accès SSH

- Problème : le serveur refusait la clé `stackflow` (« Permission denied (publickey,password) »). La clé n'avait pas été cochée lors de la création du serveur.
- Option « Rebuild » étudiée puis écartée : elle effacerait le disque sans permettre d'ajouter une clé SSH.
- Solution retenue : réinitialisation du mot de passe root depuis la console Hetzner, puis installation de la clé publique avec `ssh-copy-id`.
- Vérification : connexion SSH par clé fonctionnelle (Ubuntu 26.04.1 LTS).

```
ssh-copy-id -i ~/.ssh/stackflow.pub root@37.27.147.21
ssh -i ~/.ssh/stackflow root@37.27.147.21
```

#### Étape 4 — Déploiement de l'application

Le script `deploy.sh` automatise l'ensemble du déploiement :

| # | Action du script         | Détail                                                                                                                                                |
|---|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Installation de Docker   | Script officiel <code>get.docker.com</code> (seulement si Docker est absent)                                                                          |
| 2 | Synchronisation du code  | <code>rsync</code> vers <code>/opt/taskflow</code> (sans <code>.git</code> , <code>node_modules</code> , <code>target</code> , <code>.env...</code> ) |
| 3 | Création des secrets     | Mot de passe PostgreSQL et clé JWT aléatoires dans <code>/opt/taskflow/.env</code> (droits 600), générés une seule fois                               |
| 4 | Configuration du domaine | Ajout de <code>DOMAIN=...</code> dans le fichier <code>.env</code>                                                                                    |
| 5 | Build et démarrage       | <code>docker compose -f docker-compose.prod.yml up -d --build</code> puis nettoyage des anciennes images                                              |

```
./deploy.sh 37.27.147.21 root
```

Résultat : 3 conteneurs démarrés (db *healthy*, backend *healthy*, frontend). Spring Boot démarré en environ 16 secondes, migrations Flyway appliquées. Page d'accueil : HTTP 200 ; `/api/tasks` sans jeton : HTTP 401 (comportement attendu).

---

## Étape 5 — Tests fonctionnels (HTTP)

- Ouverture de l'application dans le navigateur Docker et vérification de la page de connexion.
- Avec le compte *hamza* : création d'une tâche, passage à « En cours », rechargement (persistance vérifiée), suppression.
- Logs nginx du serveur : requêtes POST, PATCH et DELETE sur l'API bien reçues.
- Test « live » complet avec un compte de test dédié (`claudetest@taskflow.dev`), saisie visible caractère par caractère.

## Étape 6 — Versionnement sur GitHub

Commit `e1d15d4` : ajout de `deploy.sh` et `docker-compose.prod.yml`, poussé sur la branche `main`.  
Vérification préalable : aucun mot de passe ni secret dans les fichiers commités.

## Étape 7 — Mise en place du HTTPS

- Ajout d'un service **Caddy** devant le frontend dans `docker-compose.prod.yml` (ports 80, 443 et 443/udp ; volumes pour conserver les certificats).
- Nouveau fichier `Caddyfile` : reverse proxy vers le frontend, compression gzip, et redirection de toute requête HTTP (y compris l'IP brute) vers le domaine HTTPS.
- Domaine gratuit **37-27-147-21.sslip.io** (résolution automatique vers l'IP, sans configuration DNS).
- `deploy.sh` accepte maintenant un domaine en 3<sup>e</sup> argument (par défaut : `<ip>.sslip.io`).
- Configuration validée avec `caddy validate`, puis redéploiement.

| Vérification                                          | Résultat                                                                                  |
|-------------------------------------------------------|-------------------------------------------------------------------------------------------|
| <code>https://37-27-147-21.sslip.io/</code>           | 200 — certificat Let's Encrypt (CN=37-27-147-21.sslip.io), valide jusqu'au 2 janvier 2027 |
| <code>https://.../api/tasks</code> sans jeton         | 401 (attendu)                                                                             |
| <code>http://37-27-147-21.sslip.io/tasks</code>       | 308 → <code>https://37-27-147-21.sslip.io/tasks</code>                                    |
| <code>http://37.27.147.21/login</code> (ancienne URL) | 301 → <code>https://37-27-147-21.sslip.io/login</code>                                    |
| Connexion via l'API en HTTPS                          | 200 — contexte sécurisé confirmé par le navigateur                                        |

Commit `8431d73` « Serve production over HTTPS with Caddy », poussé sur GitHub.

## Étape 8 — Test live complet en HTTPS

| Étape du test                                | Résultat                                     |
|----------------------------------------------|----------------------------------------------|
| 0. Vérification du contexte sécurisé (HTTPS) | Réussi                                       |
| 1. Déconnexion de l'utilisateur courant      | Réussi                                       |
| 2. Inscription du compte de test             | Refus attendu : « Email already registered » |
| 3–4. Connexion avec le compte de test        | Réussi — « Claude Test » connecté            |
| 5. Création d'une tâche                      | Réussi                                       |
| 6. Statut : À faire → En cours → Terminé     | Réussi                                       |
| 7. Modification du titre                     | Réussi                                       |
| 8. Rechargement : données persistées         | Réussi                                       |
| 9. Suppression de la tâche                   | Réussi                                       |
| 10. Déconnexion                              | Réussi                                       |

Une tâche résiduelle d'un test interrompu a été supprimée via l'API ; le compte de test est vide.

## Étape 9 — Nom de domaine personnalisé (en cours)

Une adresse du type `https://taskflow/tasks` n'est pas possible sur internet : un domaine nécessite une extension (.com, .org...). Option choisie : sous-domaine gratuit **DuckDNS** (ex.

`taskflow-hamza.duckdns.org`). Le site `duckdns.org` a été ouvert dans le navigateur Docker ; il reste à se connecter, créer le sous-domaine pointant vers 37.27.147.21, puis lancer :

```
./deploy.sh 37.27.147.21 root taskflow-hamza.duckdns.org
```

## 4. Fichiers de déploiement

| Fichier                                              | Rôle                                                                 |
|------------------------------------------------------|----------------------------------------------------------------------|
| <code>deploy.sh</code>                               | Déploiement en une commande : Docker, rsync, secrets, domaine, build |
| <code>docker-compose.prod.yml</code>                 | Stack de production : db, backend, frontend, caddy                   |
| <code>Caddyfile</code>                               | HTTPS automatique et redirections                                    |
| <code>frontend/nginx/default.conf.template</code>    | Sert l'application Angular et proxifie /api vers le backend          |
| <code>/opt/taskflow/.env</code> (serveur uniquement) | Secrets : DB_PASSWORD, JWT_SECRET, DOMAIN — jamais commité           |

## 5. Commandes utiles

Depuis le Mac (dossier du projet) :

```
# Redéployer après une modification du code
ssh-add ~/.ssh/stackflow
./deploy.sh 37.27.147.21 root [domaine]

# Se connecter au serveur
ssh root@37.27.147.21
```

Sur le serveur :

```
cd /opt/taskflow
docker compose -f docker-compose.prod.yml ps           # état des conteneurs
docker compose -f docker-compose.prod.yml logs -f backend # logs de l'API
docker compose -f docker-compose.prod.yml restart      # redémarrer
```

## 6. Sécurité et prochaines étapes

| Priorité | Action                                                                                                        |
|----------|---------------------------------------------------------------------------------------------------------------|
| Haute    | Changer le mot de passe root du serveur ( <code>passwd</code> ) : il a été partagé dans une conversation.     |
| Haute    | Désactiver la connexion SSH par mot de passe ( <code>PasswordAuthentication no</code> ), la clé fonctionnant. |
| Moyenne  | Finaliser le domaine DuckDNS ou acheter un domaine professionnel (.com, .dev, .ma).                           |
| Moyenne  | Activer un pare-feu (Hetzner Cloud Firewall ou <code>ufw</code> ) : n'autoriser que les ports 22, 80 et 443.  |
| Moyenne  | Mettre en place des sauvegardes de la base PostgreSQL ( <code>pg_dump</code> planifié ou backups Hetzner).    |
| Basse    | Supprimer le compte de test <code>claudetest@taskflow.dev</code> une fois les tests terminés.                 |
| Basse    | Automatiser le déploiement via GitHub Actions à chaque push sur main.                                         |